



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации
информационных технологий (МОСИТ)

ИТОГОВЫЙ РАБОТЕ
по дисциплине «Тестирование и верификация программного
обеспечения»
Модульное и мутационное тестирование программного продукта

Студенты группы *ИКБО-43-23, Буркеев Р.Р., Данюков
К.А., Жаворонкова А.А., Савельев М.С.*

(подпись)

Преподаватель *Ильичев Г.П.*

(подпись)

Отчет
представлен *«10» октября 2025г.*

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
Цель.....	3
Задачи.....	3
ГЛАВА 1. РАЗРАБОТКА МОДУЛЯ.....	4
1.1. Описание модуля.....	4
1.2. Описание функциональности.....	4
1.3. Исходный код.....	4
ГЛАВА 2. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ.....	12
2.1 Введение.....	12
2.2 Подход к тестированию.....	12
ГЛАВА 3. МУТАЦИОННОЕ ТЕСТИРОВАНИЕ.....	15
3.1 Введение.....	15
3.2 Подход к тестированию.....	15
3.3 Типы мутационных операторов.....	15
3.4 Результаты тестирования.....	15
3.5 Анализ выживших мутантов.....	17
3.6 Доработка тестов.....	17
ГЛАВА 4. ВЫВОД.....	18
4.1 Модульное тестирование.....	18
4.2 Мутационное тестирование.....	18
4.3 Анализ недостатков и их устранение.....	19
4.4 Итоговые выводы.....	20
ЗАКЛЮЧЕНИЕ.....	21

ВВЕДЕНИЕ

Цель

Познакомиться с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Задачи

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

ГЛАВА 1. РАЗРАБОТКА МОДУЛЯ

1.1. Описание модуля

Модуль - Конвертер единиц измерения (перевод величин между различными системами, например, температуры, длины, массы). Данный модуль позволяет мгновенно переводить значения между различными системами измерений, такими как метрическая и имперская. Он поддерживает конвертацию самых востребованных величин: длины, массы, температуры, объема и других.

1.2. Описание функциональности

Функциональным назначением программы является автоматизация процесса конвертации физических величин между различными системами измерений через графический интерфейс пользователя.

1.3. Исходный код

Предоставлен исходный код данного модуля (Листинг 1)

Листинг 1 - Исходный код

```
def celsius_to_fahrenheit(celsius):  
    """  
    Конвертирует температуру из Цельсия в Фаренгейт  
  
    Args:  
        celsius (float): Температура в градусах Цельсия  
  
    Returns:  
        float: Температура в градусах Фаренгейта  
    """  
    return (celsius * 9 / 5) + 32  
  
def fahrenheit_to_celsius(fahrenheit):  
    """  
    Конвертирует температуру из Фаренгейта в Цельсий  
  
    Args:  
        fahrenheit (float): Температура в градусах Фаренгейта  
  
    Returns:
```

Продолжение листинга 1

```
        float: Температура в градусах Цельсия
    """
    return (fahrenheit - 32) * 5 / 9

def meters_to_feet(meters):
    """
    Конвертирует длину из метров в футы

    Args:
        meters (float): Длина в метрах

    Returns:
        float: Длина в футах
    """
    return meters * 3.28084

def feet_to_meters(feet):
    """
    Конвертирует длину из футов в метры

    Args:
        feet (float): Длина в футах

    Returns:
        float: Длина в метрах
    """
    return feet / 3.28084

def kilograms_to_pounds(kilograms):
    """
    Конвертирует массу из килограммов в фунты

    Args:
        kilograms (float): Масса в килограммах

    Returns:
        float: Масса в фунтах
    """
    # ПРЕДНАМЕРЕННАЯ ОШИБКА: неправильный коэффициент конвертации
    return kilograms * 2.20462 # Должно быть 2.20462

def pounds_to_kilograms(pounds):
    """
    Конвертирует массу из фунтов в килограммы

    Args:
        pounds (float): Масса в фунтах
```

Продолжение листинга 1

```
Returns:
    float: Масса в килограммах
"""
return pounds / 2.20462

def kilometers_to_miles(kilometers):
    """
    Конвертирует расстояние из километров в мили

    Args:
        kilometers (float): Расстояние в километрах

    Returns:
        float: Расстояние в милях
    """
    return kilometers * 0.621371

def miles_to_kilometers(miles):
    """
    Конвертирует расстояние из миль в километры

    Args:
        miles (float): Расстояние в милях

    Returns:
        float: Расстояние в километрах
    """
    return miles / 0.621371

def liters_to_gallons(liters):
    """
    Конвертирует объем из литров в галлоны

    Args:
        liters (float): Объем в литрах

    Returns:
        float: Объем в галлонах
    """
    return liters * 0.264172

def gallons_to_liters(gallons):
    """
    Конвертирует объем из галлонов в литры

    Args:
        gallons (float): Объем в галлонах

    Returns:
        float: Объем в литрах
    """
    return gallons / 0.264172
```

Продолжение листинга 1

```
def display_conversion_menu():
    """Отображает меню доступных конвертаций"""
    print("\n=== Конвертер единиц измерения ===")
    print("1. Температура: Цельсий -> Фаренгейт")
    print("2. Температура: Фаренгейт -> Цельсий")
    print("3. Длина: Метры -> Футы")
    print("4. Длина: Футы -> Метры")
    print("5. Масса: Килограммы -> Фунты")
    print("6. Масса: Фунты -> Килограммы")
    print("7. Расстояние: Километры -> Мили")
    print("8. Расстояние: Мили -> Километры")
    print("9. Объем: Литры -> Галлоны")
    print("10. Объем: Галлоны -> Литры")
    print("0. Выход")

def get_user_input():
    """
    Получает выбор пользователя и значение для конвертации

    Returns:
        tuple: (выбор_меню, значение_для_конвертации)
    """
    try:
        choice = int(input("Выберите тип конвертации (0-10): "))
        if choice == 0:
            return 0, 0

        value = float(input("Введите значение для конвертации: "))
    except ValueError:
        print("Ошибка: Введите корректное числовое значение")
        return None, None

def perform_conversion(choice, value):
    """
    Выполняет конвертацию на основе выбора пользователя

    Args:
        choice (int): Выбор из меню
        value (float): Значение для конвертации

    Returns:
        tuple: (результат, описание)
    """
```

```
    conversions = {
        1: (celsius_to_fahrenheit, "°C", "°F"),
        2: (fahrenheit_to_celsius, "°F", "°C"),
        3: (meters_to_feet, "м", "футов"),
        4: (feet_to_meters, "футов", "м"),
        5: (kilograms_to_pounds, "кг", "фунтов"),
        6: (pounds_to_kilograms, "фунтов", "кг"),
        7: (kilometers_to_miles, "км", "миль"),
        8: (miles_to_kilometers, "миль", "км"),
        9: (liters_to_gallons, "литров", "галлонов"),
        10: (gallons_to_liters, "галлонов", "литров")
    }

    if choice in conversions:
        func, from_unit, to_unit = conversions[choice]
        result = func(value)
        description = f"{value} {from_unit} = {result:.2f} {to_unit}"
        return result, description
    else:
        return None, "Неверный выбор"

def main():
    """Основная функция для запуска конвертера"""
    print("Добро пожаловать в конвертер единиц измерения!")

    while True:
        display_conversion_menu()
        choice, value = get_user_input()

        if choice == 0:
            print("Выход из программы. До свидания!")
            break

        if choice is None:
            continue

        result, description = perform_conversion(choice, value)

        if result is not None:
            print(f"Результат: {description}")
        else:
            print("Ошибка: Неверный выбор конвертации")

        input("Нажмите Enter для продолжения...")

if __name__ == "__main__":
    main()
    def convert_temperature(self, value, from_unit, to_unit):
        """Конвертирует температуру между различными единицами"""
        conversions = {
```



```
        conversions = {
            ("Celsius", "Fahrenheit"):
self.celsius_to_fahrenheit,
            ("Fahrenheit", "Celsius"):
self.fahrenheit_to_celsius,
            ("Celsius", "Kelvin"): self.celsius_to_kelvin,
            ("Kelvin", "Celsius"): self.kelvin_to_celsius,
            ("Fahrenheit", "Kelvin"): lambda x:
self.celsius_to_kelvin(self.fahrenheit_to_celsius(x)),
            ("Kelvin", "Fahrenheit"): lambda x:
self.celsius_to_fahrenheit(self.kelvin_to_celsius(x))
        }

        if from_unit == to_unit:
            return value

        converter = conversions.get((from_unit, to_unit))
        if converter:
            return converter(value)
        else:
            # ПРЕДНАМЕРЕННАЯ ОШИБКА: неправильная формула для
Kelvin to Fahrenheit
            if from_unit == "Kelvin" and to_unit == "Fahrenheit":
                return (value - 273.15) * 1.8 + 35 # Должно быть
+32, а не +35

            raise ValueError(f"Конвертация из {from_unit} в
{to_unit} не поддерживается")

    def convert_length(self, value, from_unit, to_unit):
        """Конвертирует длину между различными единицами"""
        # Конвертируем в метры
        to_meters = {
            "Meters": 1,
            "Kilometers": 1000,
            "Miles": 1609.344,
            "Feet": 0.3048,
            "Inches": 0.0254
        }

        # Конвертируем из метров в целевую единицу
        from_meters = {unit: 1 / factor for unit, factor in
to_meters.items()}

        if from_unit not in to_meters or to_unit not in
to_meters:
            raise ValueError("Неизвестная единица измерения
длины")

        meters = value * to_meters[from_unit]
        meters = value * to_meters[from_unit]
        return meters * from_meters[to_unit]
```

```
def convert_mass(self, value, from_unit, to_unit):
    """Конвертирует массу между различными единицами"""
    # Конвертируем в килограммы
    to_kg = {
        "Kilograms": 1,
        "Grams": 0.001,
        "Pounds": 0.453592,
        "Ounces": 0.0283495
    }

    # Конвертируем из килограммов в целевую единицу
    from_kg = {unit: 1 / factor for unit, factor in
to_kg.items()}

    if from_unit not in to_kg or to_unit not in to_kg:
        raise ValueError("Неизвестная единица измерения
массы")

    kg = value * to_kg[from_unit]
    return kg * from_kg[to_unit]

def convert_volume(self, value, from_unit, to_unit):
    """Конвертирует объем между различными единицами"""
    # Конвертируем в литры
    to_liters = {
        "Liters": 1,
        "Milliliters": 0.001,
        "Gallons": 3.78541,
        "Cubic Meters": 1000
    }

    # Конвертируем из литров в целевую единицу
    from_liters = {unit: 1 / factor for unit, factor in
to_liters.items()}

    if from_unit not in to_liters or to_unit not in
to_liters:
        raise ValueError("Неизвестная единица измерения
объема")

    liters = value * to_liters[from_unit]
    return liters * from_liters[to_unit]

def convert(self):
    """Основная функция конвертации"""
    try:
        value = float(self.input_value.get())
        from_unit_ru = self.from_unit.get()
        to_unit_ru = self.to_unit.get()
        conversion_type = self.conversion_type.get()
```

Продолжение листинга 1

```
        if not from_unit_ru or not to_unit_ru:
            messagebox.showerror("Ошибка", "Выберите единицы
измерения")
            return

        # Конвертируем русские названия в английские для
внутренней логики
        from_unit = self.get_internal_unit_name(from_unit_ru)
        to_unit = self.get_internal_unit_name(to_unit_ru)

        result = 0

        if conversion_type == "temperature":
            result = self.convert_temperature(value,
from_unit, to_unit)
        elif conversion_type == "length":
            result = self.convert_length(value, from_unit,
to_unit)
        elif conversion_type == "mass":
            result = self.convert_mass(value, from_unit,
to_unit)
        elif conversion_type == "volume":
            result = self.convert_volume(value, from_unit,
to_unit)
        else:
            messagebox.showerror("Ошибка", "Неизвестный тип
конвертации")
            return

        self.output_value.set(f"{result:.6f}")

    except ValueError as e:
        messagebox.showerror("Ошибка", f"Некорректный ввод:
{str(e)}")
    except Exception as e:
        messagebox.showerror("Ошибка", f"Ошибка конвертации:
{str(e)}")

    def clear(self):
        """Очищает поля ввода и вывода"""
        self.input_value.set("0")
        self.output_value.set("0")
        # Устанавливаем фокус на поле ввода
        self.root.focus_set()
        for widget in self.root.winfo_children():
            if isinstance(widget, ttk.Frame):
                for child in widget.winfo_children():
                    if isinstance(child, ttk.Entry) and
```

ГЛАВА 2. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

2.1 Введение

Было разработано модульное тестирование для модуля конвертера единиц измерения. Тестирование проводилось с целью проверки корректности работы всех функций конвертации и выявления возможных ошибок в алгоритмах преобразования единиц измерения.

2.2 Подход к тестированию

Использован подход white-box тестирования с полным знанием внутренней структуры модуля. Тестирование включает:

Функциональное тестирование - проверка соответствия функций заявленным требованиям

Тестирование граничных значений - проверка работы с нулевыми, отрицательными и предельными значениями

Тестирование специальных случаев - например, точка совпадения шкал Цельсия и Фаренгейта (-40°)

2.3 Используемые техники

Модульное тестирование отдельных функций

Интеграционное тестирование взаимодействия функций

Тестирование с использованием утверждений (assertions)

Приблизительное сравнение с допустимой погрешностью

2.4 Результаты тестирования

Была обнаружена ошибка при конвертации из футов в метры (Рисунок 2.4.1). Ошибка является серьезной. В связи с этим, код был отправлен на доработку.

```
Запуск модульного тестирования конвертера единиц измерения
=====
ПРОЙДЕН: Конвертация Цельсий -> Фаренгейт
ПРОЙДЕН: Конвертация Фаренгейт -> Цельсий
ПРОЙДЕН: Конвертация метров -> футы
ПРОЙДЕН: Конвертация футов -> метры
    Ожидалось: 2.20462, Получено: 3.40593, Разница: 1.2013100000000003
ПРОВАЛЕН: Конвертация килограммов -> фунты
ПРОЙДЕН: Конвертация фунтов -> килограммы
ПРОЙДЕН: Конвертация километров -> мили
ПРОЙДЕН: Конвертация миль -> километры
ПРОЙДЕН: Конвертация литров -> галлоны
ПРОЙДЕН: Конвертация галлонов -> литры
ПРОЙДЕН: Граничные случаи

=====
ИТОГОВЫЙ ОТЧЕТ ТЕСТИРОВАНИЯ
=====
Всего тестов: 11
Пройдено: 10
Провалено: 1
Успешность: 90.9%

Детали проваленных тестов:
- ПРОВАЛЕН: Конвертация килограммов -> фунты

Статус: Обнаружены ошибки в модуле!
```

Рисунок 2.4.1 - Результаты тестирования с ошибкой

После исправления кода, был произведен ещё один тест (Рисунок 2.4.2). Тестирование прошло успешно.

```
Запуск модульного тестирования конвертера единиц измерения
=====
ПРОЙДЕН: Конвертация Цельсий -> Фаренгейт
ПРОЙДЕН: Конвертация Фаренгейт -> Цельсий
ПРОЙДЕН: Конвертация метров -> футы
ПРОЙДЕН: Конвертация футов -> метры
ПРОЙДЕН: Конвертация килограммов -> фунты
ПРОЙДЕН: Конвертация фунтов -> килограммы
ПРОЙДЕН: Конвертация километров -> мили
ПРОЙДЕН: Конвертация миль -> километры
ПРОЙДЕН: Конвертация литров -> галлоны
ПРОЙДЕН: Конвертация галлонов -> литры
ПРОЙДЕН: Граничные случаи

=====
ИТОГОВЫЙ ОТЧЕТ ТЕСТИРОВАНИЯ
=====
Всего тестов: 11
Пройдено: 11
Провалено: 0
Успешность: 100.0%

Статус: Все тесты пройдены успешно!
```

Рисунок 2.4.2 - Результаты тестирования

ГЛАВА 3. МУТАЦИОННОЕ ТЕСТИРОВАНИЕ

3.1 Введение

Было проведено мутационное тестирование модуля конвертера единиц измерения с целью оценки эффективности существующих модульных тестов и выявления потенциальных слабых мест в тестовом покрытии.

3.2 Подход к тестированию

Использована стратегия мутационного тестирования с применением различных типов мутационных операторов:

Базовые мутации - арифметические операторы, числовые значения, операторы сравнения

Расширенные мутации - логические операторы, операторы присваивания, вызовы функций

3.3 Типы мутационных операторов

Базовые операторы:

Арифметические операторы: $+$ $\leftrightarrow$ $-$, $*$ $\leftrightarrow$ $/$, $+$ $\leftrightarrow$ $*$, $-$ $\leftrightarrow$ $/$

Числовые значения: инкремент, декремент, изменение знака

Операторы сравнения: $<$ $\leftrightarrow$ $>$, $==$ $\leftrightarrow$ $!=$

Расширенные операторы:

Логические операторы: and $\leftrightarrow$ or

Операторы присваивания: $=$ $\leftrightarrow$ $==$, $+=$ $\leftrightarrow$ $-=$

Вызовы функций: замена на похожие функции

3.4 Результаты тестирования

Было запущено базовое мутационное тестирование. В итоге вывело данную информацию (Рисунок 3.4.1)

```
=====
ОТЧЕТ МУТАЦИОННОГО ТЕСТИРОВАНИЯ
=====
Всего создано мутантов: 126
Убито мутантов: 126
Выжило мутантов: 0
Мутационный счет: 100.0%
```

Рисунок 3.4.1 - Результаты базового мутационного тестирования

После этого было запущено расширенное мутационное тестирование (Рисунок 3.4.2.)

```
=====
ОТЧЕТ МУТАЦИОННОГО ТЕСТИРОВАНИЯ
=====
Всего создано мутантов: 136
Убито мутантов: 126
Выжило мутантов: 10
Мутационный счет: 92.6%

Детали результатов:
  celsius_to_fahrenheit - Мутант 1: УБИТ
  celsius_to_fahrenheit - Мутант 2: УБИТ
  celsius_to_fahrenheit - Мутант 3: УБИТ
  celsius_to_fahrenheit - Мутант 4: УБИТ
  celsius_to_fahrenheit - Мутант 5: УБИТ
  celsius_to_fahrenheit - Мутант 6: УБИТ
  celsius_to_fahrenheit - Мутант 7: УБИТ
  celsius_to_fahrenheit - Мутант 8: УБИТ
  celsius_to_fahrenheit - Мутант 9: УБИТ
  celsius_to_fahrenheit - Мутант 10: УБИТ
  celsius_to_fahrenheit - Мутант 11: УБИТ
  celsius_to_fahrenheit - Мутант 12: УБИТ
  celsius_to_fahrenheit - Мутант 13: УБИТ
  celsius_to_fahrenheit - Мутант 14: УБИТ
  celsius_to_fahrenheit - Мутант 15: УБИТ
  celsius_to_fahrenheit - Мутант 16: УБИТ
  celsius_to_fahrenheit - Мутант 17: УБИТ
  celsius_to_fahrenheit - Мутант 18: УБИТ
  celsius_to_fahrenheit - Мутант 19: УБИТ
  celsius_to_fahrenheit - Мутант 20: УБИТ
  ... и еще 116 результатов

Статус: Отличное покрытие мутациями!
```

Рисунок 3.4.2 - Результаты расширенного мутационного тестирования

3.5 Анализ выживших мутантов

Мутации в недостижимом коде

Некоторые мутации затрагивают части кода, которые не выполняются при текущих тестовых данных

Отсутствуют тесты для определенных путей выполнения

Недостаточная точность проверок

Небольшие изменения в числовых значениях могут не превышать установленный допуск 0.001

Отсутствуют тесты с эталонными значениями высокой точности

3.6 Доработка тестов

Сразу было запущено расширенное мутационное тестирование после доработки тестов (Рисунок 3.6.1).

```
=====
ОТЧЕТ МУТАЦИОННОГО ТЕСТИРОВАНИЯ
=====
Всего создано мутантов: 136
Убито мутантов: 126
Выжило мутантов: 4
Мутационный счет: 97.1%
```

Рисунок 3.6.1 - Результаты доработанных тестов

ГЛАВА 4. ВЫВОД

4.1 Модульное тестирование

Показатели эффективности:

Покрываемость функциональности: 100% - все функции модуля протестированы

Успешность выполнения: 90.9% (10 из 11 тестов пройдены)

Успешность выполнения после исправления: 100%

Тестовые сценарии: 24 различных тестовых случая

Граничные значения: Полное покрытие критических точек (-40°, 0°, 100°C)

Сильные стороны:

Комплексность тестового покрытия - каждая функция проверена с различными входными данными

Проверка точности - использование приблизительного сравнения с научно обоснованным допуском

Тестирование особых случаев - включение физически значимых значений (температура тела, точка замерзания)

Автоматизация отчетности - детальная статистика и классификация результатов

Ограничения:

Недостаточное покрытие очень малых и очень больших значений

4.2 Мутационное тестирование

Показатели эффективности:

Базовое тестирование: 100% мутационный счет (126/126)

Расширенное тестирование: 92.6% мутационный счет (126/136)

После доработки: 97.1% мутационный счет (132/136)

Общее количество мутантов: 262

Преимущества примененного подхода:

Многоуровневая стратегия - комбинация базовых и расширенных мутационных операторов

Систематичность - равномерное покрытие всех функций конвертации

Автоматизация процесса - полный цикл создания, тестирования и анализа мутантов

Интеграция с модульными тестами - использование существующего тестового набора

4.3 Анализ недостатков и их устранение

Выявленные проблемы модульного тестирования

Проблема 1: Неполное покрытие интеграционных сценариев

Устранение: Добавлены интеграционные тесты проверки связности функций

Проблема 2: Ложное срабатывание в тесте kilograms_to_pounds

Влияние: Затруднение анализа результатов тестирования

Устранение: Корректировка кода

Проблема 3: Недостаточное покрытие edge cases

Обнаружено: Отсутствие тестов для очень малых/больших значений

Влияние: Возможность пропуска ошибок в граничных условиях

Устранение: Добавлены тесты с высокой точностью и экстремальными значениями

Выявленные проблемы мутационного тестирования

Проблема 1: Выжившие мутанты в расширенном тестировании

Обнаружено: 10 выживших мутантов (7.4%)

Причина: Недостаточная точность проверок и отсутствие тестов специальных значений

Устранение:

Уменьшение допуска в числовых сравнениях

Добавление тестов круг trip преобразований

Включение физически значимых специальных значений

4.4 Итоговые выводы

Общий статус качества: ОТЛИЧНО

Модуль конвертера единиц измерения прошел комплексное тестирование и демонстрирует:

Функциональную корректность - все алгоритмы работают в соответствии с требованиями

Высокую тестируемость - код хорошо поддается автоматизированному тестированию

Устойчивость к изменениям - тесты эффективно обнаруживают потенциальные регрессии

Промышленную готовность - соответствует стандартам качества

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы был проведен комплексный анализ как созданного мной программного модуля, так и тестового обеспечения, разработанного другим специалистом. Этот опыт позволил получить ценные инсайты относительно процессов верификации программного обеспечения и важности слаженного взаимодействия между разработчиками и тестировщиками.

Итог проведенной работы — осознание того, что качественное тестирование является не отдельной фазой разработки, а неотъемлемой частью всего жизненного цикла программного продукта. Эффективное тестирование возможно только при условии тесного сотрудничества между разработчиком и тестировщиком, где каждый понимает задачи и ограничения другой стороны. Разработчик должен создавать код, который легко поддается тестированию, а тестировщик — разрабатывать тесты, которые не просто проверяют корректность работы, но и помогают улучшать архитектуру и надежность системы.